



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: Grupo 7

No de Práctica(s): Práctica 5

Integrante(s): 323064039

426058744

120002850

323071378

323214636

426059576

Semestre: 2027 - 1

Fecha de entrega: 2 de Octubre de 2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. Abstracción y encapsulamiento	2
2.2. Constructores y estado inicial	2
2.3. Operaciones de los ejercicios	3
3. Desarrollo	3
3.1. Ejercicio 1: Modificadores dentro de constructores	3
3.2. Ejercicio 2: Empleados y aumento salarial	9
3.3. Ejercicio 3: representación de una fecha	13
4. Resultados	17
4.1. Empleados: Salarios positivos	17
4.2. Empleados: salarios no positivos	17
4.3. Fecha: visualización de los componentes	17
5. Conclusiones	19

1. Introducción

La práctica aborda la abstracción y el encapsulamiento mediante tres ejercicios: analizar si conviene llamar métodos modificadores desde los constructores, representar dos empleados y sus salarios, y representar una fecha. El programa de empleados debe mostrar el salario anual de cada objeto antes y después de un aumento del 10 %. El programa de fecha debe mostrar mes, día y año separados por diagonales y asumir que los datos proporcionados son correctos.

Separar los datos de un objeto de la interacción con el usuario permite concentrar sus reglas en una clase. En este caso, esa separación impide que un salario no positivo sustituya el valor almacenado y permite mostrar una fecha mediante una operación propia del objeto. Además el análisis de los constructores ayuda a distinguir la inicialización del objeto de sus cambios posteriores.

1.1. Objetivos

- Justificar el uso de modificadores en constructores.
- Implementar las clases Empleado y Fecha con sus atributos privados, y utilizarlas desde MainEmpleado y MainFecha.
- Comprobar el cálculo anual, el aumento salarial y la representación de la fecha mediante casos deterministas, manteniendo una solución sencilla con constructores, métodos y decisiones básicas.

2. Marco Teórico

2.1. Abstracción y encapsulamiento

La abstracción selecciona los datos pertinentes para una tarea: nombre, apellido y salario mensual para un empleado; mes, día y año para una fecha. Por otro lado, el encapsulamiento concentra esos datos y sus operaciones en una clase.

Los atributos privados se utilizan desde la propia clase y los métodos públicos ofrecen operaciones accesibles desde las clases principales. Asimismo, un método de consulta (getters) devuelve un dato y un modificador actualiza un atributo (setters).[1]

2.2. Constructores y estado inicial

Un constructor tiene el nombre de su clase, no declara un tipo de retorno y participa en la creación de un objeto, los parámetros recibidos se utilizan para establecer su estado inicial. La expresión `this` permite distinguir un atributo del parámetro con el mismo nombre.[1] Es importante resaltar que antes de las asignaciones del constructor,

Java inicializa los valores en 0, por ejemplo, un atributo de tipo `double` tiene el valor predeterminado 0.0.[3]

2.3. Operaciones de los ejercicios

El salario anual se calcula como $S_a = 12S_m$. Un aumento del 10% produce $S'_m = 1.10S_m$; después se calcula $S'_a = 12S'_m$. La condición $S_m > 0$ determina si se acepta una asignación salarial.

La fecha se representa mediante la concatenación de sus tres enteros, en el orden mes/día/año. El enunciado no solicita comprobar meses, años bisiestos ni límites de días.

3. Desarrollo

3.1. Ejercicio 1: Modificadores dentro de constructores

Llamar un modificador desde un constructor es válido en Java, pero su conveniencia depende de lo que haga ese método, si una misma regla debe aplicarse al crear un objeto y al modificarlo después, reutilizarla puede evitar que las dos operaciones acepten valores diferentes, como en este caso ya que, un empleado debe rechazar un salario no positivo tanto al comenzar como al recibir una actualización, sin embargo, la reutilización solo es adecuada cuando la operación puede ejecutarse con el estado disponible durante la construcción; no basta con que el método funcione después de crear el objeto.

Durante la construcción todavía pueden faltar asignaciones de atributos, es por ello que un modificador que dependa de otros datos podría consultar valores iniciales en lugar de los definitivos, así como la posibilidad de generar errores, este riesgo se incluye para justificar la respuesta de investigación.[2]

Para estas clases se elige asignar directamente los atributos en los constructores, en `Empleado` se comprueba que el salario sea positivo antes de guardarlo; el modificador mantiene la misma condición para los cambios posteriores, si el valor inicial no se acepta, el atributo permanece en 0.0 por la inicialización predeterminada de Java.[3]

En `Fecha` se guardan directamente los tres enteros ya que se asumen datos correctos, repetición de una condición sencilla facilita seguir cada operación sin añadir métodos auxiliares, por tanto, usar modificadores en un constructor no es una obligación ni un error por sí mismo: debe evaluarse la regla.

Esto se sustenta en documentación de Java; queda pendiente contrastar las respuestas de Luis, Zoé, Alex, Joseph, Hanniel y Alan. Las figuras siguientes reservan su evidencia. Incorporar las capturas no completa por sí solo la comparación escrita.

La Figura 3.1 corresponde a la consulta de Luis.

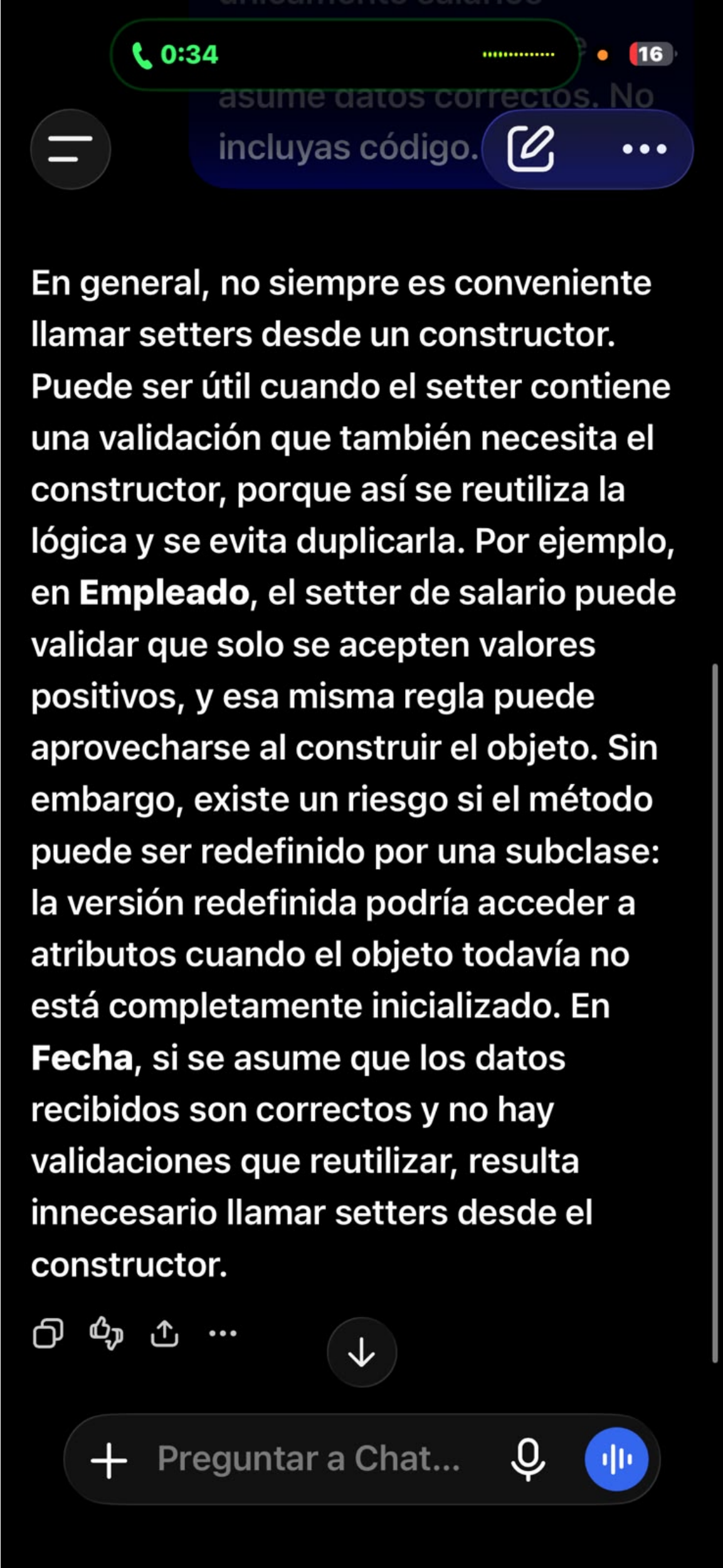


Figura 3.1: Consulta y respuesta de investigación de Luis

La Figura 3.2 corresponde a la consulta de Zoé.

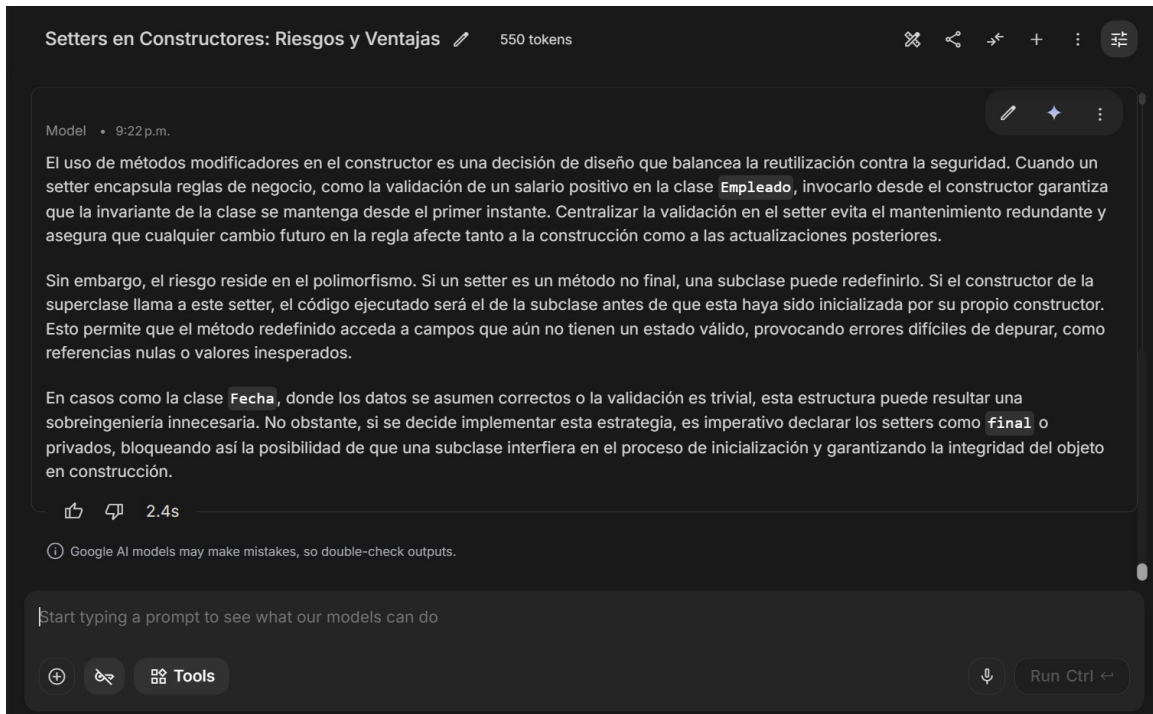


Figura 3.2: Consulta y respuesta de investigación de Zoé

La Figura 3.3 corresponde a la consulta de Alex.

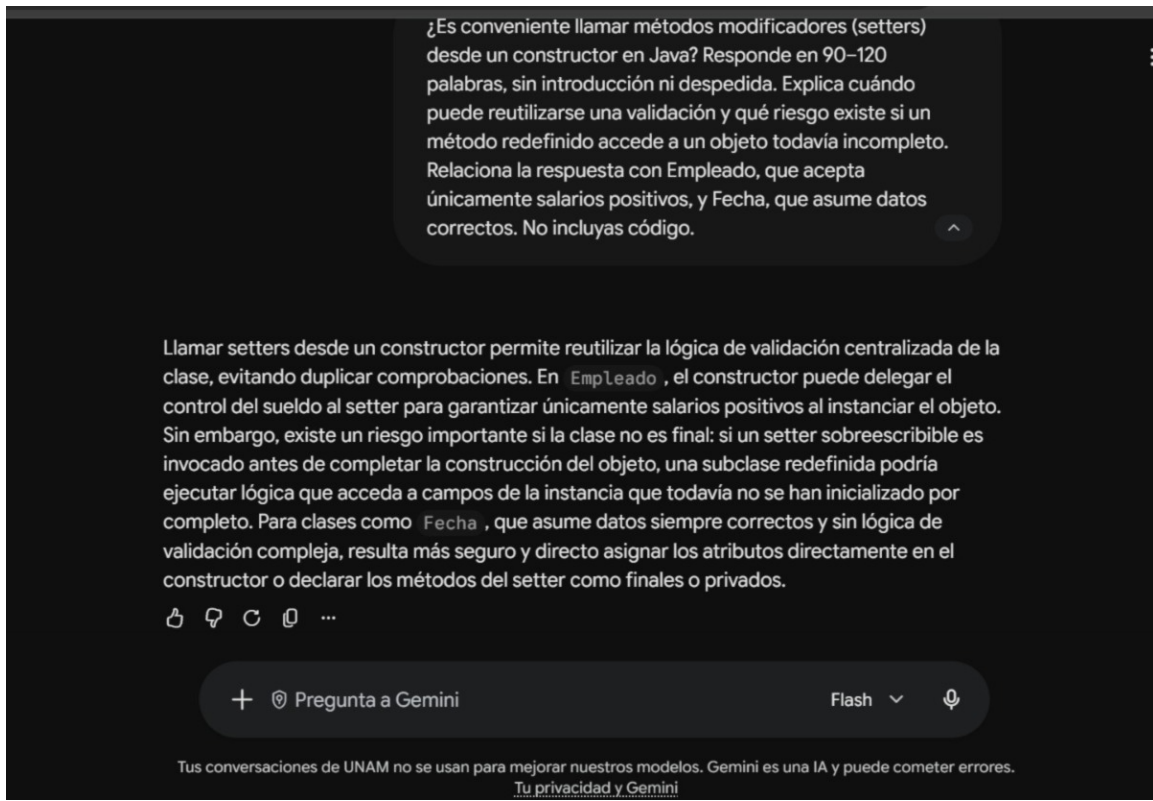


Figura 3.3: Consulta y respuesta de investigación de Alex

La Figura 3.4 corresponde a la consulta de Joseph.

 aistudio.xiaomimimo.com/#/ultra/ee6c 

¿Es conveniente llamar métodos modificadores (setters) desde un constructor en Java? Responde en menos de 50 palabras de forma súper breve y concreta



MiMo-V2.6-Pro UltraSpeed

Thought for 0.4 seconds 

No. Si la clase es extendible, el setter puede estar sobrescrito y ejecutarse sobre una subclase aún no inicializada; además puede disparar validaciones o eventos inesperados. Asigna los campos directamente (o usa un método privado o final) y deja los setters para uso externo.

Thinking TPS

644 tokens/s

0.4s · 385 tokens

Figura 3.4: Consulta y respuesta de investigación de Joseph

La Figura 3.5 corresponde a la consulta de Hanniel.

¿Es conveniente llamar métodos modificadores (setters) desde un constructor en Java? Responde en 90–120 palabras, sin introducción ni despedida. Explica cuándo puede reutilizarse una validación y qué riesgo existe si un método redefinido accede a un objeto todavía incompleto. Relaciona la respuesta con Empleado, que acepta únicamente salarios positivos, y Fecha, que asume datos correctos. No incluyas código.

Llamar setters desde un constructor puede ser conveniente cuando permiten **reutilizar una validación necesaria para mantener la consistencia del objeto**. Por ejemplo, en `Empleado`, si el salario solo puede ser positivo, el constructor podría aprovechar la misma validación utilizada posteriormente al modificarlo. Sin embargo, existe un riesgo importante si el setter puede ser redefinido mediante herencia: la versión redefinida podría ejecutarse antes de que el objeto termine de construirse y acceder a atributos todavía no inicializados, provocando resultados incorrectos o excepciones. Por ello, suele preferirse validar directamente en el constructor. En `Fecha`, si se asume que los datos recibidos son correctos, llamar setters aporta poco beneficio y añade dependencias innecesarias durante la construcción.



Figura 3.5: Consulta y respuesta de investigación de Hanniel

La Figura 3.6 corresponde a la consulta de Alan.

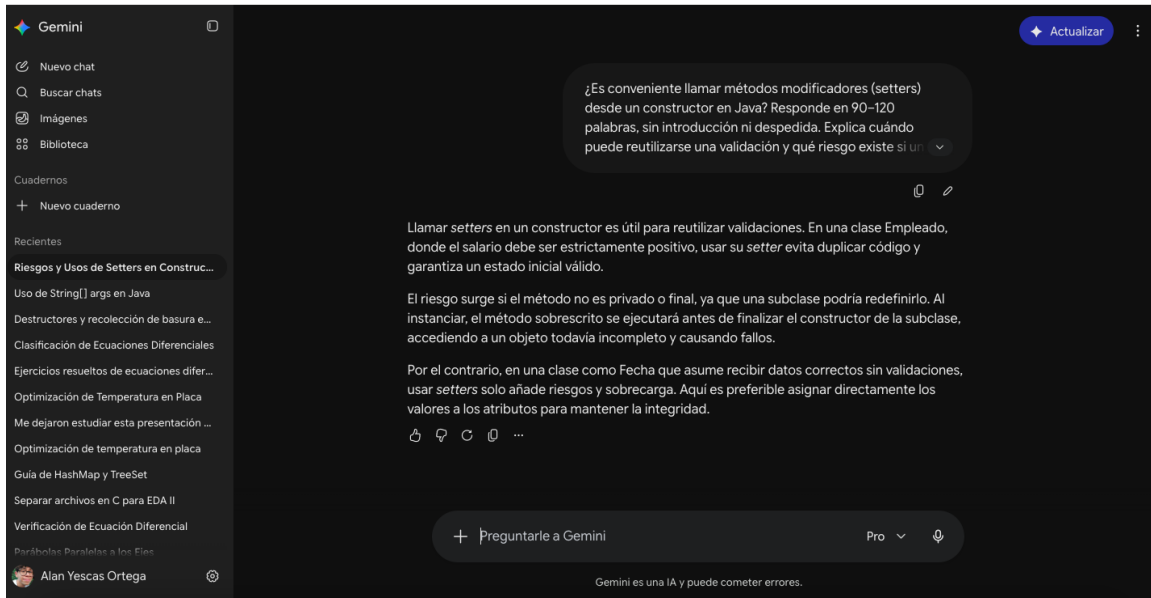


Figura 3.6: Consulta y respuesta de investigación de Alan

3.2. Ejercicio 2: Empleados y aumento salarial

Empleado contiene los atributos privados `nombre`, `apellido` y `salarioMensual`, su constructor recibe los tres datos y los modificadores de nombre y apellido asignan el nuevo texto, asimismo, sus consultas devuelven el dato almacenado. El modificador salarial solo asigna el argumento si es mayor que cero, si se rechaza una actualización, conserva el salario anterior; si se rechaza el valor inicial, permanece en 0.0.

MainEmpleado lee nombres y apellidos completos con `Scanner` y convierte los datos salariales a `double`. Crea dos objetos independientes y calcula cada salario anual consultando su salario mensual y multiplicándolo por doce. Después solicita la actualización mensual con el factor 1.10 y vuelve a mostrar los salarios anuales. La clase principal coordina la entrada y la presentación; Empleado conserva los datos y controla las asignaciones.

La Figura 3.7 representa la modificación salarial.

La Figura 3.8 reúne los atributos y las operaciones reales de Empleado. La Figura 3.9 muestra el uso de los dos objetos desde MainEmpleado, antes y después del aumento.

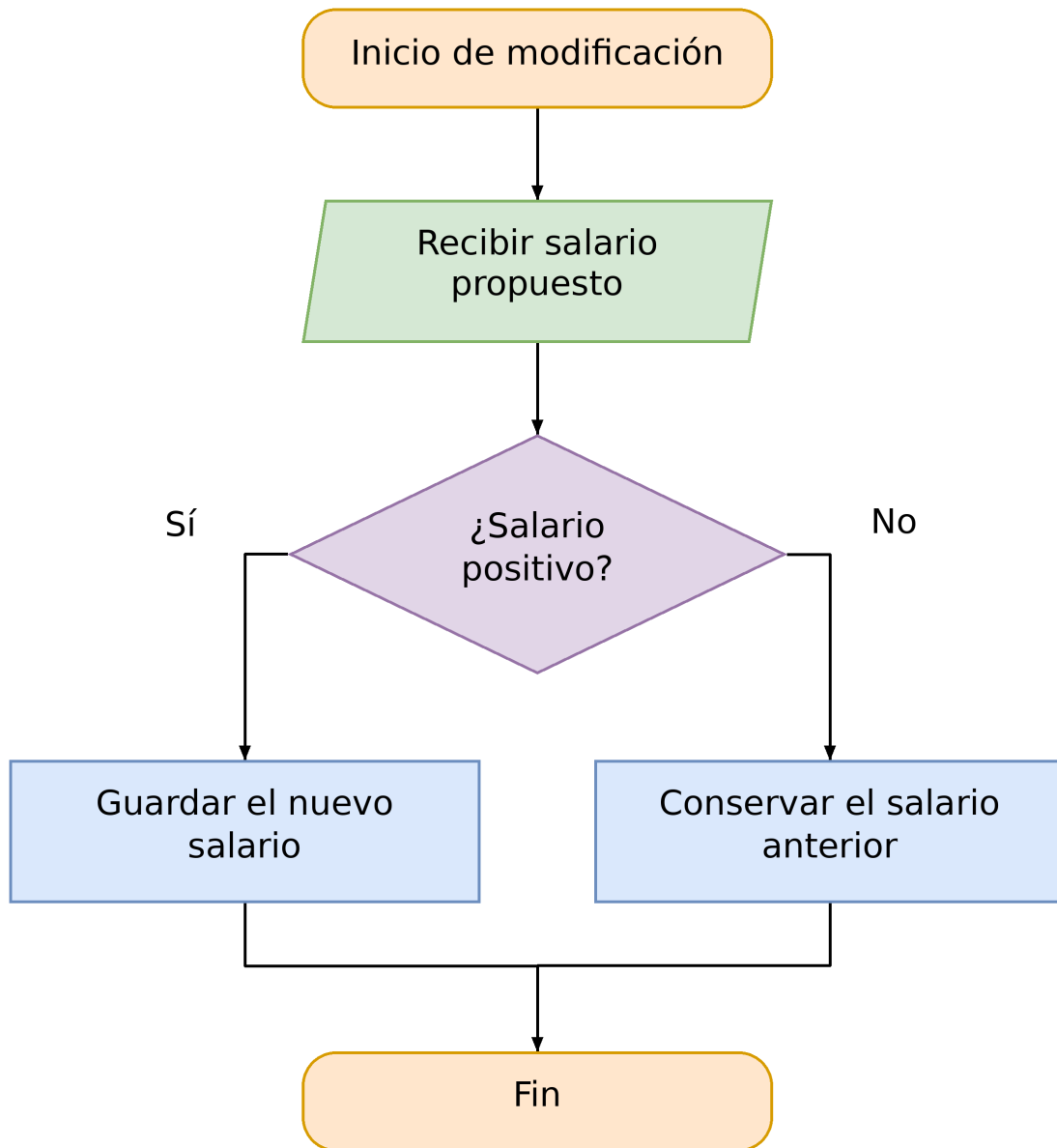


Figura 3.7: Flujo de la modificación salarial en Empleado

Empleado
- nombre : String - apellido : String - salarioMensual : double
+ Empleado(nombre : String, apellido : String, salarioMensual : double) + getNombre() : String + setNombre(nombre : String) : void + getApellido() : String + setApellido(apellido : String) : void + getSalarioMensual() : double + setSalarioMensual(salarioMensual : double) : void

Figura 3.8: Tabla de la clase Empleado

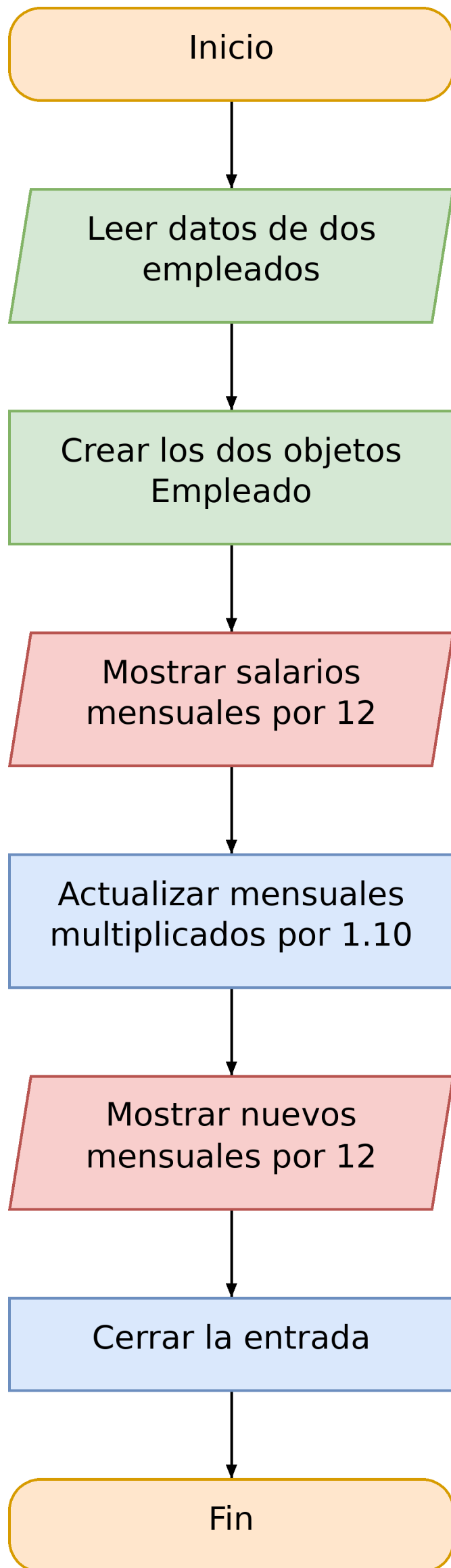


Figura 3.9: Flujo de MainEmpleado

La Figura 3.10 muestra que MainEmpleado declara únicamente el método principal; sus datos de lectura son variables locales.

MainEmpleado
<u>+ main(args : String[]) : void</u>

Figura 3.10: Tabla de la clase MainEmpleado

3.3. Ejercicio 3: representación de una fecha

Fecha contiene tres atributos privados de tipo `int`: `mes`, `dia` y `anio`, el constructor los inicializa directamente mientras que los modificadores `setMes`, `setDia` y `setAnio` permiten cambiar cada componente por separado; `mostrarFecha` imprime los valores separados por diagonales.

MainFecha solicita los tres enteros, crea el objeto y llama a `mostrarFecha`, la clase principal utiliza Fecha sin acceder directamente a sus atributos privados, para esta prueba se utiliza el constructor y la visualización; no se inventa una secuencia que obligue a ejecutar los tres modificadores antes de mostrar el objeto.

La Figura 3.11 representa la visualización de la fecha almacenada. La Figura 3.12 presenta los atributos, el constructor y los métodos explícitos de Fecha. La Figura 3.13 muestra la lectura, construcción y utilización de Fecha desde MainFecha.

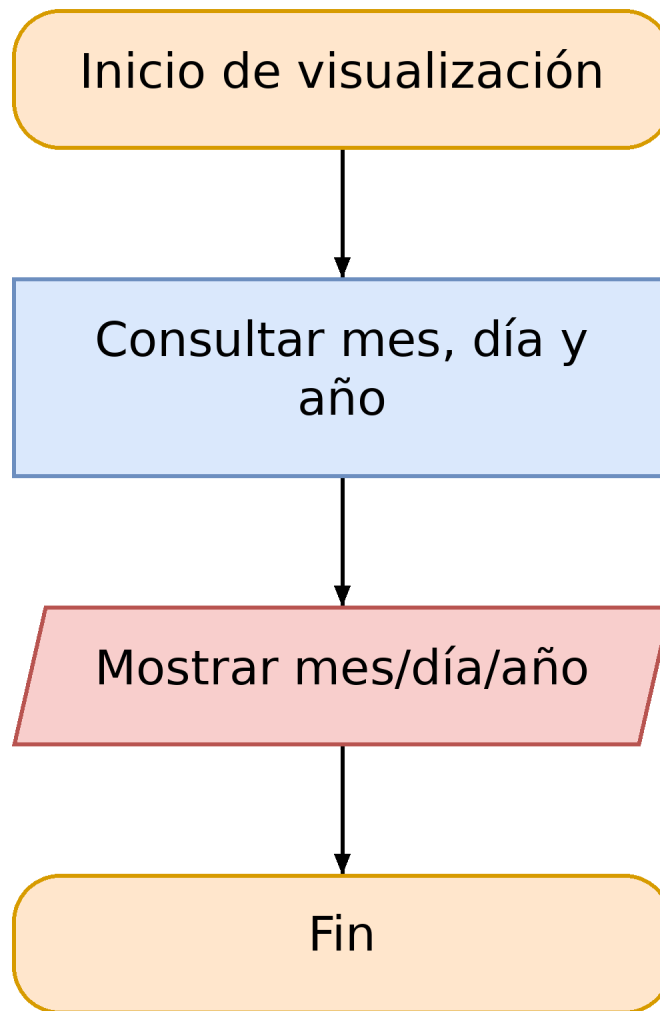


Figura 3.11: Flujo de mostrarFecha en la clase Fecha

Fecha
- mes : int - dia : int - anio : int
+ Fecha(mes : int, dia : int, anio : int) + setMes(mes : int) : void + setDia(dia : int) : void + setAnio(anio : int) : void + mostrarFecha() : void

Figura 3.12: Tabla de la clase Fecha

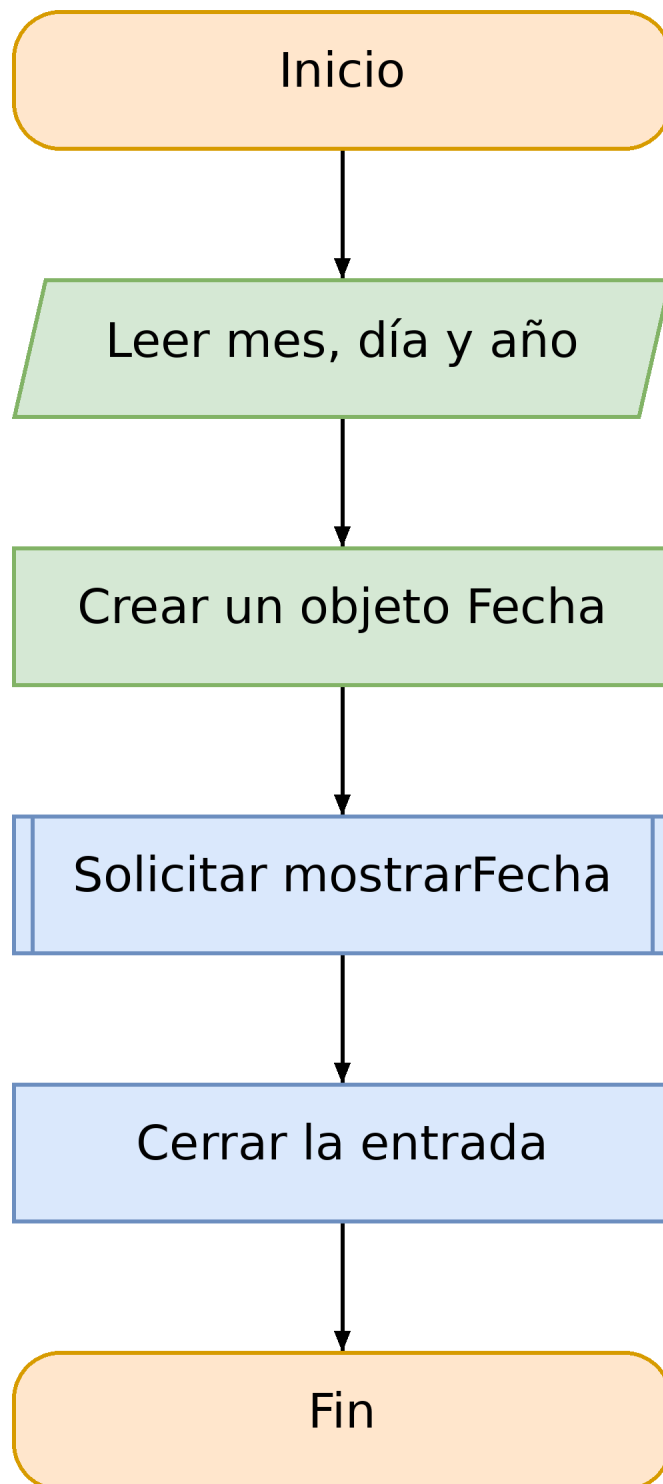


Figura 3.13: Flujo de MainFecha

La Figura 3.14 presenta el método principal de MainFecha, sin atribuirle las variables locales como atributos.

MainFecha
<u>+ main(args : String[]) : void</u>

Figura 3.14: Tabla de la clase MainFecha

4. Resultados

4.1. Empleados: Salarios positivos

Se utilizan Ana Perez con salario mensual 10000 y Luis Lopez con salario mensual 15000, con salarios anuales de 120000 y 180000 respectivamente, después del aumento del 10 %, los mensuales son 11000 y 16500, y los anuales son 132000 y 198000. La consola muestra estas cantidades con terminación decimal número.0 por su tipo `double`. La Figura 4.1 presenta la evidencia de este caso.

4.2. Empleados: salarios no positivos

El segundo caso utiliza Ana Perez con salario mensual propuesto de `-500` y Luis Lopez con salario propuesto de `0`. Ninguno satisface la condición de aceptación, por lo que los atributos permanecen en `0.0`. Los salarios anuales también son `0.0` y el intento de aumento conserva ese valor, el programa no asigna el número negativo, no solicita otro dato y no imprime un mensaje de error, así como la Figura 4.2 muestra la diferencia respecto al caso positivo.

4.3. Fecha: visualización de los componentes

Las entradas son mes 10, día 2 y año 2026. La salida de la fecha es `10/2/2026`, con el mes primero y sin relleno de ceros, representado en la Figura 4.3 la cual presenta la lectura y la representación del objeto.

```
Datos del primer empleado:
Nombre: Ana
Apellido: Perez
Salario mensual (usa punto para los decimales): 10000

Datos del segundo empleado:
Nombre: Luis
Apellido: Lopez
Salario mensual (usa punto para los decimales): 15000
Salario anual de Ana: 120000.0
Salario anual de Luis: 180000.0
Salario anual con aumento de Ana: 132000.0
Salario anual con aumento de Luis: 198000.0
```

Figura 4.1: Salarios anuales antes y después del aumento

```
Datos del primer empleado:
Nombre: Ana
Apellido: Perez
Salario mensual (usa punto para los decimales): -500

Datos del segundo empleado:
Nombre: Luis
Apellido: Lopez
Salario mensual (usa punto para los decimales): 0
Salario anual de Ana: 0.0
Salario anual de Luis: 0.0
Salario anual con aumento de Ana: 0.0
Salario anual con aumento de Luis: 0.0
```

Figura 4.2: Conservación del valor inicial ante salarios no positivos

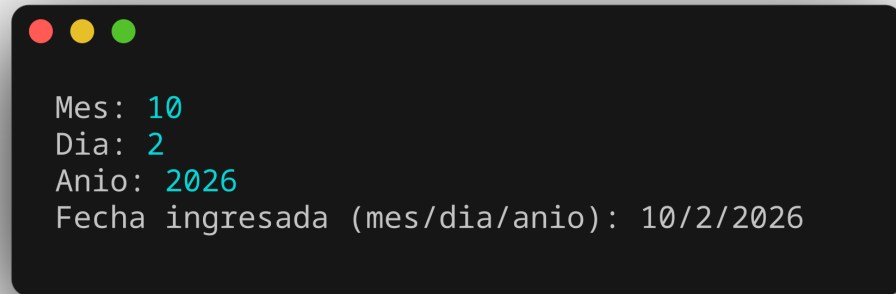


Figura 4.3: Representación de la fecha en formato mes/día/año

5. Conclusiones

La condición salarial permite mantener separado el dato propuesto del dato aceptado, así mismo los casos positivos muestran que el aumento se aplica al salario mensual y que su anualización conserva el mismo incremento proporcional, mientras que los casos no positivos muestran que rechazar una asignación conserva el estado previo; al construir un objeto, ese estado corresponde al valor predeterminado del atributo.

La representación de una fecha requiere conservar el orden de sus componentes y utilizar una operación de la clase para mostrarlos, es así como el encapsulamiento permite que las clases principales trabajen con objetos mediante sus métodos, así la inicialización directa mantiene visible lo que se establece al construirlos, mientras que los modificadores definen los cambios posteriores, reutilizar estos métodos desde un constructor debe decidirse según sus dependencias y sus efectos, sin asumir que siempre es conveniente.

Referencias

- [1] Oracle. *The Java Language Specification, Java SE 25 Edition*. Capítulo 8: Classes; secciones 8.3, 8.4 y 8.8. <https://docs.oracle.com/javase/specs/jls/se25/html/jls-8.html>.
- [2] Oracle. *The Java Language Specification, Java SE 25 Edition*. Sección 12.5: Creation of New Class Instances. <https://docs.oracle.com/javase/specs/jls/se25/html/jls-12.html#jls-12.5>.

- [3] Oracle. *The Java Language Specification, Java SE 25 Edition*. Sección 4.12.5: Initial Values of Variables. <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html#jls-4.12.5>.